Caracteres em C

- Entrada de caracteres

```
scanf("%c", &x);
x = getchar();
```

- Saída de caracteres

```
printf("%c", x);
putchar(x);
```

```c
#include <stdio.h>

int main(){
  char x;

  printf("\ndigite um caractere: ");
  x = getchar();

  printf("\no caractere digitado eh: ");
  putchar(x);

  printf("\n\n");
  return 0;
}
```

```
digite um caractere: s

o caractere digitado eh: s
```

Strings em C

- vetor de caracteres com terminador '\0'
- Ex: string **str** de 9 posições, com as posições de 0 a 6 ocupadas (demais posições com "lixo")

	0	1	2	3	4	5	6	7	8
str	B	r	a	s	i	l	\0	?	?

Declaração:

```
char  <identificador> [tamanho+1];
```

Strings: inicialização

- `char nome[15] = “Ivo”;`

  insere os caracteres entre “” na string nome, a partir da posição 0 e, ao final, acrescenta ‘\0’ (no ex. posição 3)

- `char nome[15] = {´I´, ´v´, ´o´};`

  insere os caracteres entre {} a partir da posição 0. Se o tamanho da string > nro de caracteres armazenados, as demais posições são preenchidas com zeros

- `char nome[] = “Ivo”;  // *nome`

  determina nro de caracteres entre “”, soma 1 e cria a string com esse tamanho

Entrada e saída de strings – uso do scanf() e printf()

- Entrada

```
scanf("%s", s1);
```

sem &

- Saída

```
printf("%s", s1);
```

scanf – leitura até espaço em branco

```
#include <stdio.h>

int main(){
  char s1[20];

  printf("\ndigite uma string: ");
  scanf("%s", s1);

  printf("\na string eh: %s", s1);

  printf("\n\n");
  return 0;
}
```

```
digite uma string: Ola mundo

a string eh: Ola
```

Entrada e saída de strings: uso do gets(), fgets() e puts()

- Entrada

```
gets(s1);
```

Warning → gets

```
fgets(s1,10,stdin);
```

```c
#include <stdio.h>

int main(){
  char s1[20];

  printf("\ndigite uma string: ");
  gets(s1);

  printf("\na string eh: ");
  puts(s1);

  printf("\n\n");
  return 0;
}
```

```
digite uma string: ola mundo

a string eh: ola mundo
```

- Saída

```
puts(s1);
```

```c
#include <stdio.h>

int main(){
  char s1[20];

  printf("\ndigite uma string: ");
  fgets(s1, 10, stdin);

  printf("\na string eh: ");
  puts(s1);

  printf("\n\n");
  return 0;
}
```

```
digite uma string: ola mundo

a string eh: ola mundo
```

Entrada e saída de strings: caractere por caractere

```c
#include <stdio.h>

int main(){
  char s1[20];
  int i;

  printf("\n\ndigite uma string: ");
  fgets(s1, sizeof(s1), stdin);

  for(i=0; s1[i] !='\0'; i++)
      printf("%c \n", s1[i]);

  printf("\n\\");
  return 0;
}
```

```
digite uma string: ola mundo
o
l
a

m
u
n
d
o
```

Manipulação de strings

- String não é um tipo primitivo da linguagem C, logo as seguintes operações **NÃO** são válidas:

```
str1 = str2;          // ERRO! Não copia str2 em str1
if (str1 == str2)     // ERRO! Não compara str1 com str2
```

- Porém, seus elementos individuais admitem:

```
str[3] = 'b';
if (str[0] == 'a')
```

Funções para manipulação de strings

- **Biblioteca string.h**

Função strcpy (string.h)

Copia a *string_origem* na *string_destino*

```
strcpy(string_destino, string_origem);
```

```c
#include <stdio.h>
#include <string.h>

int main(){
  char s1[10], s2[10];


  printf("\n\ndigite 2 palavras: ");
  fgets(s1, sizeof(s1), stdin);
  fgets(s2, sizeof(s2), stdin);

  printf("\nANTES\n string s1: %s string s2: %s", s1, s2);
  strcpy(s2, s1);
  printf("\nAPOS\n string s1: %s string s2: %s", s1, s2);

  printf("\n\n");
  return 0;
}
```

```
digite 2 palavras: cao
gato

ANTES
 string s1: cao
 string s2: gato

APOS
 string s1: cao
 string s2: cao
```

Função strcat (string.h)

Anexa a *string_origem* ao final da *string_destino*

```
strcat(string_destino, string_origem);
```

```c
#include <stdio.h>
#include <string.h>

int main(){
  char s1[10], s2[20];


  printf("\n\ndigite 2 palavras: ");
  scanf("%s", s1);
  scanf("%s", s2);

  printf("\nANTES\n string s1: %s\n string s2: %s", s1, s2);
  strcat(s2, s1);
  printf("\nAPOS\n string s1: %s\n string s2: %s", s1, s2);

  printf("\n\n");
  return 0;
}
```

Tamanho da s2 deve ser compatível com s1

```
digite 2 palavras: cao
gato

ANTES
 string s1: cao
 string s2: gato
APOS
 string s1: cao
 string s2: gatocao
```

Função strlen (string.h)

Retorna o tamanho de uma string (despreza o '\0')

```
strlen(str);
```

```c
#include <stdio.h>
#include <string.h>

int main(){
  char s1[10];
  int tam;

  printf("\n\ndigite uma palavra: ");
  scanf("%s", s1);

  tam = strlen(s1);
  printf("\nstring %s tem tamanho %d\n", s1, tam);

  printf("\n\n");
  return 0;
}
```

```
digite uma palavra: caderno

string caderno tem tamanho 7
```

Indo além… outras funções (string.h)

- **strstr**(str1, str2) — retorna um ponteiro para primeira ocorrência de str2 em str1
- **strchr**(str1, ch) — retorna um ponteiro para primeira ocorrência de ch em str1
- **strcmp**(str1, str2) – compara duas strings e:

  retorna 0 caso str1 seja igual a str2

  retorna < 0 caso str1 seja menor que str2

  retorna > 0 caso str1 seja maior que str2

- **strtok**, **strncpy**, **strncat**, **strupr**, **strlwr**...

Exercícios

Obs: nos exercícios 1, 2 e 4 não podem ser usadas funções da biblioteca string.h

1) Faça uma função que leia uma string, e 2 caracteres c1 e c2. A função deve substituir todas as ocorrências do caractere c1 por c2. Ao final, escreva as 2 strings.
2) Faça um programa que leia um caractere C e uma string S. O programa deve armazenar, em um vetor V, os índices onde c1 aparece em S. Ao final, imprima o vetor.
3) Faça uma função booleana que receba uma string e identifique se ela é um palíndromo (ex: “RADAR” e “SAIAS” são palíndromos).
4) Faça um programa que leia 10 palavras, verifique o tamanho de cada uma, armazenando os tamanhos em um vetor. Ao final escreva o vetor, a maior palavra e seu tamanho.
5) Faça um programa que leia uma string de até 30 caracteres, gere uma cópia e substitua, na cópia, os espaços em branco por *. Ao final, escreva a cópia o número de posições alteradas.

Exercícios

5) Um dos métodos de encriptação mais antigos é o de Júlio César: se uma letra a ser criptografada é a letra N do alfabeto, substitua-a com a letra N+K, onde K é um número inteiro constante (César usava K = 3). Ex.: para K = 1, a string “Adoro programar em C” se torna “Bepsp! qsphsbnbs!fn!D”. Faça um programa que leia uma string e um valor K e criptografe a string utilizando o método de César.

6) Faça uma função que receba 2 strings e um valor inteiro, representando uma posição. A seguir, insira a segunda string na primeira, na posição indicada pelo valor.

7) Faça uma função que receba 2 strings (A e B) e retorne uma terceira string (C) formada pelos caracteres de A e B intercalados. Ex.: Se A='Quarta' e B='Segunda', a resposta deve ser 'Qsueagrutnada'. A seguir, faça uma outra função que dadas as strings C e B, retorne a string A.